

AI-Powered Teaching Assistant

Technical Documentation

Submitted by: Karthik Lagudu

Problem Statement: 1 - AI-Powered Teaching Assistant

Program: AU Campus Recruitment 2026

Tech Stack: Python | Streamlit | Groq API | Llama 3.3-70b

Date: February 2026

This document describes the approach, dataset, model architecture, assumptions, limitations, and sample outputs of the AI-Powered Teaching Assistant system.

Table of Contents

1.	Approach Explanation	3
2.	Dataset Source & Generation	4
3.	Model Architecture	5
4.	Gamification & Interactive Features	7
5.	Assumptions and Limitations	9
6.	Sample Outputs	10
7.	System Design & Project Structure	11

1. Approach Explanation

The AI-Powered Teaching Assistant is designed as a modular, production-ready system with two primary intelligence engines. The architecture separates concerns between Natural Language Processing (understanding what the student needs) and Adaptive Learning (recommending what to do next).

1.1 Student Query Understanding (Q1)

Goal: Interpret natural language questions from students and extract structured metadata including intent, topic, and difficulty level.

Method: We utilize a Large Language Model (Llama-3.3-70b-versatile) via the Groq API, with carefully engineered system and user prompts. The LLM acts as an educational classifier, providing deep contextual understanding that goes beyond simple keyword matching. It can differentiate between a student asking for an 'Explanation' versus requesting an 'Example', even when the phrasing is ambiguous.

The system prompt establishes the AI persona as an EdTech assistant and constrains output to structured JSON format. The user prompt wraps the student's question with explicit classification instructions for three dimensions: Intent (Explanation, Example, Doubt clarification, Revision), Topic (e.g., Neural Networks, Optimization), and Difficulty Level (Beginner, Intermediate, Advanced).

1.2 Adaptive Learning Path Recommendation (Q2)

Goal: Recommend the next logical learning step based on student performance metrics.

Method: A hybrid approach combining rule-based pedagogical logic with LLM reasoning. The system evaluates a student's state (quiz score, number of attempts, time spent, last topic studied) against curriculum-aware rules:

- Score < 50: Action = Revision, Difficulty = Decrease, Stay on same topic
- 50 ≤ Score < 80: Action = Practice, Difficulty = Same, Reinforce current topic
- Score ≥ 80: Action = Next Topic, Difficulty = Increase, Advance in curriculum

These rules are embedded within the LLM prompt along with the full curriculum order, allowing the model to make contextually intelligent recommendations that follow a research-backed spaced-repetition and mastery-learning pedagogy.

2. Dataset Source & Generation

2.1 Source: Synthetic Data (Self-Generated)

No publicly available dataset perfectly matched the multi-turn interaction model required for adaptive learning path recommendations. Therefore, we built a custom synthetic data generator to simulate realistic student interactions.

2.2 Generation Logic

The generator script (data/synthetic_data_generator.py) creates a configurable dataset of student interactions with the following characteristics:

Field	Type	Description
student_id	Integer	Unique student identifier (1 to N)
topic	String	Randomly selected from 7 ML topics
quiz_score	Float	Normal distribution (mean=70, SD=15), clipped 0-100
attempts	Integer	Inversely correlated with score (1-4)
time_spent_minutes	Integer	Random, 5-60 minutes
last_asked_intent	String	Random: Explanation/Example/Doubt/Revision

2.3 Dataset Statistics

Default configuration: 100 students, each with 5-20 interactions, producing approximately 1,250 records. The current generated file contains 1,282 records.

Privacy Note: The dataset is entirely synthetic and contains no Personally Identifiable Information (PII). All student IDs are arbitrary integers.

2.4 Sample Data

Student	Topic	Score	Attempts	Time (min)	Intent
1	Linear Regression	68.66	1	15	Doubt clarification
1	Backpropagation	56.24	1	39	Revision
1	Neural Networks	57.51	3	9	Revision
1	Optimization	84.52	1	50	Revision
2	CNNs	94.46	1	39	Example
2	Gradient Descent	100.0	1	43	Revision

3. Model Architecture

3.1 Core Model

Model: Llama-3.3-70b-versatile (70 billion parameters)

Provider: Groq API (cloud-hosted inference)

Response Format: Structured JSON via json_object mode

3.2 Architecture Overview

The system follows a prompt-engineering-first architecture where the LLM serves as the core reasoning engine. Rather than fine-tuning a model or training from scratch, we leverage the pre-trained capabilities of Llama-3.3-70b through carefully designed prompts that encode domain knowledge and pedagogical rules.

Why Prompt Engineering? For an EdTech assistant with a well-defined scope, prompt engineering provides: (1) Zero training cost, (2) Instant iteration on logic, (3) Transparent reasoning that can be audited and improved, (4) Consistent JSON output via Groq's json_object mode.

3.3 Processing Pipeline

Query Understanding Pipeline:

Pipeline: Student Query Understanding

```
User Question (string)
  |
  v
StudentQueryEngine.analyze_query(query)
  |
  v
System Prompt: "You are an AI assistant that classifies
                student queries in an EdTech context."
User Prompt:   "Analyze: <query>. Classify into intent,
                topic, difficulty_level."

  |
  v
Groq API -> Llama-3.3-70b (json_object mode)
  |
  v
Structured JSON: { intent, topic, difficulty_level }
```

Adaptive Learning Pipeline:

Pipeline: Adaptive Learning Recommendation

```
Student State (dict: score, attempts, topic, time)
|
v
AdaptiveLearningEngine.recommend_path(state)
|
v
System Prompt: "You are an AI academic advisor."
User Prompt:   "State: <metrics>. Rules: <pedagogical
                rules>. Curriculum: <ordered topics>."
|
v
Groq API -> Llama-3.3-70b (json_object mode)
|
v
Structured JSON: { next_topic, action, difficulty_adjustment }
```

3.4 Key Design Decisions

- JSON-only output mode eliminates parsing errors and guarantees structured responses
- System prompts define persona boundaries, preventing off-topic responses
- Curriculum sequence is embedded in the prompt, enabling context-aware topic progression
- Class-based engine design allows easy swap of the underlying model or API provider

4. Gamification & Interactive Features

The application transforms passive learning into active engagement through multiple game-based learning modes, XP rewards, and achievement systems.

4.1 XP & Level System

Students earn XP through all interactions. Level progression spans 10 tiers from 'Curious Learner' (Level 1) to 'AI Grandmaster' (Level 10), requiring increasing XP thresholds (100, 250, 500, 1000, ... XP).

Action	XP Reward
Correct quiz answer	+15 XP
Attempted quiz answer	+3 XP
Memory pair matched	+10 XP
Cue card studied	+3 XP
Game completion bonus	+25 to 75 XP
Perfect score bonus	+25 XP

4.2 Memory Match Game

A card-matching game with a 4x3 grid of 12 face-down cards (6 concept-definition pairs per topic). Students flip two cards at a time; matching concept-definition pairs stay revealed. The game tracks moves and awards a 1-5 star rating based on efficiency. 42 unique pairs across 7 ML/AI topics.

- Match: Concept card (purple) + Definition card (blue)
- +10 XP per match, +25-75 XP on completion based on move count
- 5 stars for <= 8 moves, 1 star for 16+ moves

4.3 Cue Cards (Flip Cards)

Traditional flip-style study cards for active recall. Front shows the concept name, back reveals the definition. Students navigate with Previous/Flip/Next buttons. Each flip earns +3 XP. Uses the same topic and pairs as the Memory Game.

4.4 Daily Quiz

5-question adaptive quizzes generated by the LLM. Features animated correct/wrong feedback, streak tracking, and a score reveal screen with stars and XP breakdown.

4.5 Badges & Study Roadmap

- Achievement badges: First Steps, Quiz Master, Knowledge Explorer, etc.
- Daily streak tracking with fire icon indicator
- Visual learning roadmap through 7 ML/AI topics with sequential unlocking

5. Assumptions and Limitations

4.1 Assumptions

- The student follows a fixed Machine Learning curriculum in a predefined order: Linear Regression -> Gradient Descent -> Neural Networks -> Backpropagation -> Optimization Algorithms -> CNNs -> RNNs.
- Quiz scores are the primary metric for gauging student understanding and determining the appropriate next step.
- The Groq API is available with consistent uptime and acceptable latency for real-time interactions.
- Students interact with one topic at a time, and each interaction is independent (stateless per request).
- The LLM consistently follows the structured output format specified in the system/user prompts.

4.2 Limitations

LLM Dependency: The system requires an active internet connection to query the Groq API. Offline usage is not supported. Any API downtime directly impacts availability.

- Statelessness: Each request is processed independently. The system does not maintain a persistent student profile across sessions. A production deployment would require a database layer (e.g., PostgreSQL, MongoDB) to track learning history over time.
- Fixed Curriculum: The topic list is hardcoded in the prompt for this demo. A production system would need a dynamic curriculum management module.
- No Fine-Tuning: The model is used as-is via prompt engineering. Domain-specific fine-tuning could improve accuracy for edge cases but would increase cost and complexity.
- Single Language: Currently supports English-language queries only.
- No Authentication: The Streamlit demo does not implement user authentication or session management.

6. Sample Outputs

5.1 Student Query Understanding (Q1)

Sample Input: "I don't understand backpropagation."

Output JSON

```
{
  "intent": "Explanation",
  "topic": "Backpropagation",
  "difficulty_level": "Intermediate"
}
```

Sample Input: "Can you show me an example of gradient descent with code?"

Output JSON

```
{
  "intent": "Example",
  "topic": "Gradient Descent",
  "difficulty_level": "Intermediate"
}
```

Sample Input: "What is linear regression?"

Output JSON

```
{
  "intent": "Explanation",
  "topic": "Linear Regression",
  "difficulty_level": "Beginner"
}
```

5.2 Adaptive Learning Path Recommendation (Q2)

Case 1 - Low Score (Needs Revision):

Parameter	Value
Quiz Score	45
Attempts	3
Last Topic	Backpropagation
Time Spent	45 minutes

Recommendation Output

```
{
  "next_topic": "Backpropagation",
  "action": "Revision",
  "difficulty_adjustment": "Decrease"
}
```

Case 2 - High Score (Ready to Advance):

Parameter	Value
Quiz Score	92
Attempts	1
Last Topic	Neural Networks
Time Spent	30 minutes

Recommendation Output

```
{
  "next_topic": "Backpropagation",
  "action": "Next Topic",
  "difficulty_adjustment": "Increase"
}
```

Case 3 - Medium Score (Practice):

Parameter	Value
Quiz Score	65
Attempts	2
Last Topic	Gradient Descent
Time Spent	50 minutes

Recommendation Output

```
{
  "next_topic": "Gradient Descent",
  "action": "Practice",
  "difficulty_adjustment": "Same"
}
```

7. System Design & Project Structure

7.1 Project Structure

Project Tree

```
think/
|
+-- app.py                # Streamlit web interface (7 tabs)
+-- main.py               # CLI interface
+-- generate_pdf.py       # PDF documentation generator
+-- requirements.txt      # Python dependencies
+-- .env                  # API key (GROQ_API_KEY)
+-- technical_documentation.md # Source documentation
|
+-- src/
|   +-- __init__.py
|   +-- query_engine.py   # Student Query Understanding engine
|   +-- adaptive_engine.py # Adaptive Learning Path engine
|   +-- gamification.py   # XP, levels, badges, streaks
|   +-- flashcards.py     # MCQ data + Memory pairs
|   +-- styles.py         # CSS styling for web UI
|
+-- data/
|   +-- synthetic_data_generator.py # Data generation script
|   +-- student_interactions.csv    # Generated dataset
|
+-- tests/
|   +-- verify.py          # Verification / integration tests
|
+-- presentation/
|   +-- index.html         # Presentation slides
```

7.2 Module Responsibilities

Module	Purpose	Key Class/Function
query_engine.py	NLP intent/topic/difficulty classification	StudentQueryEngine
adaptive_engine.py	Learning path recommendation	AdaptiveLearningEngine
gamification.py	XP, levels, badges, streaks	add_xp(), check_badges()
flashcards.py	MCQ + Memory pairs + Cue cards	get_deck(), get_memory_pairs()
styles.py	CSS styling for all UI components	get_css()
app.py	Interactive web dashboard (7 tabs)	get_engines()
main.py	Command-line demo interface	main()
generate_pdf.py	PDF documentation generator	build_pdf()

7.3 Technology Stack

Component	Technology	Purpose
LLM	Llama-3.3-70b-versatile	Core AI reasoning
API Provider	Groq	Fast LLM inference (cloud)
Web Framework	Streamlit	Interactive UI dashboard
Language	Python 3.x	Primary programming language
Data Processing	Pandas + NumPy	Dataset handling
Config	python-dotenv	Environment variable management
PDF Generation	fpdf2	Technical documentation output

7.4 Scalability Considerations

The modular class-based architecture supports straightforward scaling:

- Engine classes can be imported into any Python web framework (Flask, FastAPI, Django)
- The Groq API handles model hosting and GPU infrastructure
- Batch processing demonstrates multi-student throughput capability
- Adding new topics or intents requires only prompt modifications, not code changes
- Flashcard data is decoupled from UI logic, enabling easy content expansion